

# Elements of a Django Web Application

Computer Science 412  
Boston University

Aaron Stevens

# Learning Objectives

- Create a new Django project and app using `django-admin` and `manage.py` commands
- Use `manage.py runserver` to start a development server locally
- Set up a GitHub repository and connect it to your local Django project
- Understand the workflow for deploying a Django app to `csa2.bu.edu`
- Explain the Model-View-Controller (MVC) architecture and how Django maps to it
- Describe the purpose of each file in a Django project directory structure

# Virtual Environments

**A virtual environment is an isolated Python installation for one project.**

- Each project can have its own set of packages — separate from system Python
- Prevents version conflicts: Project A needs Django 4.2, Project B needs Django 5.0
- Always create and activate a virtual environment before installing anything
- If you install packages globally (without a venv), conflicts will eventually break things



You must activate the virtual environment every time you open a new terminal.

# Setting Up a Virtual Environment

Run these commands once when starting a new project:

```
pip install pipenv  
pipenv shell  
pip install django
```

## What each command does:

- `pip install pipenv` — installs the virtual environment `pipenv`
- `pipenv shell` — tells your shell to use the venv Python and pip
- `pip install django` — downloads and installs Django only inside this venv

# Setting Up a Virtual Environment

Run these commands once when starting a new project:

```
python -m venv venv  
source venv/bin/activate  
pip install django
```

## What each command does:

- `python -m venv venv` — creates a folder called `venv/` with an isolated Python
- `source venv/bin/activate` — tells your shell to use the `venv` Python and `pip`
- `pip install django` — downloads and installs Django only inside this `venv`

 On Windows, activate with: `venv\Scripts\activate`

# Creating a Django Project

Run once at the start — creates the top-level project skeleton:

```
django-admin startproject cs412 .
```

## What this creates:

- manage.py — command-line utility for running the server, migrations, and more
- mysite/settings.py — all configuration: database, installed apps, secret key
- mysite/urls.py — the root URL routing table for the whole project
- mysite/wsgi.py — entry point for a production web server

 The trailing dot (.) puts the files in the current directory rather than a new subfolder.

# Creating a Django App

A project can contain many apps. Run this for each new feature area:

```
python manage.py startapp myapp
```

## What this creates (inside myapp/):

- views.py — Python functions/classes that handle requests and return responses
- models.py — Python classes that define database tables
- admin.py — registers models with the Django admin interface
- apps.py — app configuration (used in INSTALLED\_APPS in settings.py)
- migrations/ — tracks database schema changes over time

 After creating an app, add it to INSTALLED\_APPS in settings.py.

# Django File System

Computer Science 412  
Boston University

Aaron Stevens

# The Django Directory Structure

```
▼ django
  ▼ cs412
    📄 __init__.py
    📄 asgi.py
    📄 settings.py
    📄 urls.py
    📄 wsgi.py
    > formdata
  ▼ hw
    > migrations
    > templates
    📄 __init__.py
    📄 admin.py
    📄 apps.py
    📄 models.py
    📄 tests.py
    📄 urls.py
    📄 views.py
    > static
    ≡ db.sqlite3
    📄 manage.py
    ≡ Pipfile
    {} Pipfile.lock
    📄 Procfile
    ⓘ README.md
```

The Django directory contains configuration, scripts, and subdirectories for each app.  
The project directory contains configurations project-level URLs and settings.

The project-level **settings.py** file contains configurations settings.

The project-level **urls.py** file contains URL mapping for the entire project.

The **templates** directory contains HTML template files used by the generic views.

The app-level **urls.py** file contains URL-to-view mappings for one app.

The app-level **views.py** file contains view functions or classes to respond to a URL request.

The **manage.py** file is used to run the development server, as well as administrate commands.

The **Pipfile** contains the requirements of the virtual environment.

# The Django Directory Structure

```
▼ django
  ▼ cs412
    __init__.py
    asgi.py
    settings.py
    urls.py
    wsgi.py
  > formdata
  ▼ hw
    > migrations
    > templates
    __init__.py
    admin.py
    apps.py
    models.py
    tests.py
    urls.py
    views.py
  > static
  db.sqlite3
  manage.py
  Pipfile
  Pipfile.lock
  Procfile
  README.md
```

The Django directory contains configuration, scripts, and subdirectories for each app.  
The project directory contains configurations project-level URLs and settings.

The project-level **settings.py** file contains configurations settings.

The project-level **urls.py** file contains URL mapping for the entire project.

The **migrations** directory tracks changes in the database.

The **templates** directory contains HTML template files used by the generic views.

The **admin.py** file is used to register model classes with the built-in Django admin tool.

The **models.py** file is used to create model classes, which represent data objects in Django.

The app-level **urls.py** file contains URL-to-view mappings for one app.

The app-level **views.py** file contains view functions or classes to respond to a URL request.

The **db.sqlite3** file contains the database used by Django to store data records.

The **manage.py** file is used to run the development server, as well as administrate commands.

The **Pipfile** contains the requirements of the virtual environment.

# Running the Development Server

Start a local web server to test your app in the browser:

```
python manage.py runserver
```

## What happens:

- Django starts a web server at `http://127.0.0.1:8000/` (localhost)
- Changes to `.py` files are detected automatically — no restart needed
- The server runs until you stop it with `Ctrl + C`
- Only use this server for development — it is not for production use

 You must activate your virtual environment before running this command.

# Introduction to Source Code Version Control and GitHub

Computer Science 412  
Boston University

Aaron Stevens

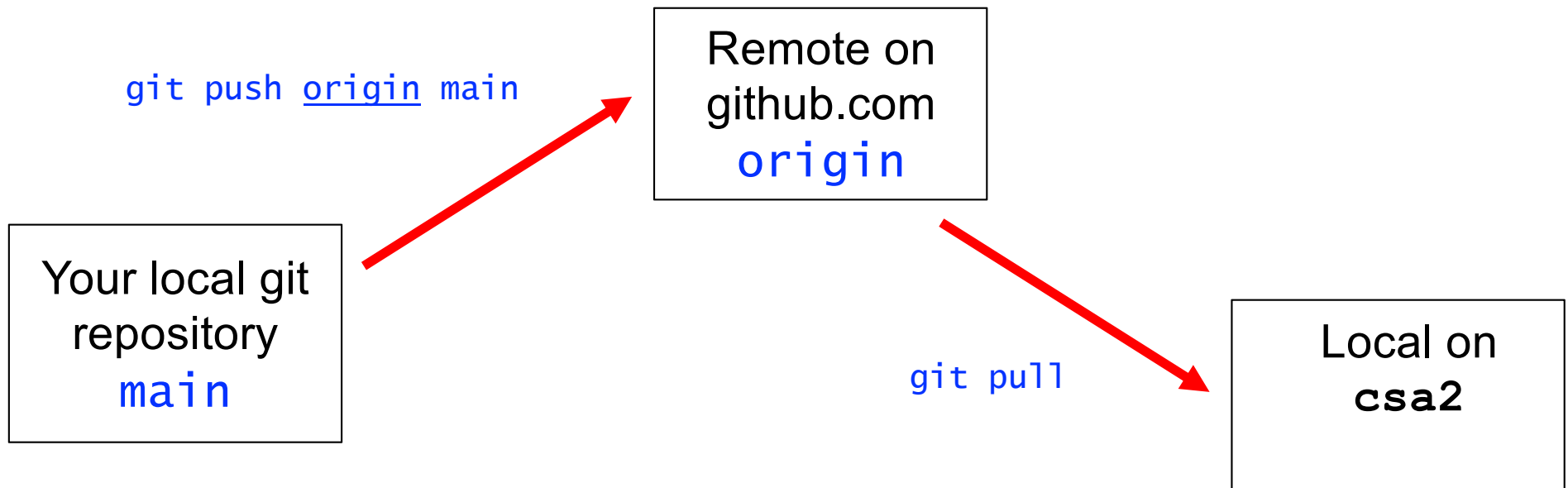
# What is Version Control?

- The product of your programming is **source code**.
- **Version control** provides a way to track multiple versions of your code.
- As you develop code, you “commit” changes with comments.
  - This provides multiple **backups** of your code.
  - You can view each version, compare versions, and restore from an old version (if needed).

# What is Git and GitHub?

- Git is the industry-standard version control tracking software.
- GitHub is a cloud-based git repository – where you store your code. GitHub provides:
  - storage
  - tools for viewing/sharing/collaborating
- We will also use git for deployment to the **cs-webapps.bu.edu** server.

## Relationship between: Local files, GitHub, Gradescope, and csa2



### Reminder: what are all these parts?

Local repository: the files on your computer, where you develop/test code

GitHub repository: a version-controlled backup of your files

Gradescope: submit files here for grading

csa2: linux computer cluster on which you have a CS computing account.

cs-webapps: webserver that serves your web application on public www.

# Elements of a Django Web Application

Computer Science 412  
Boston University

Aaron Stevens

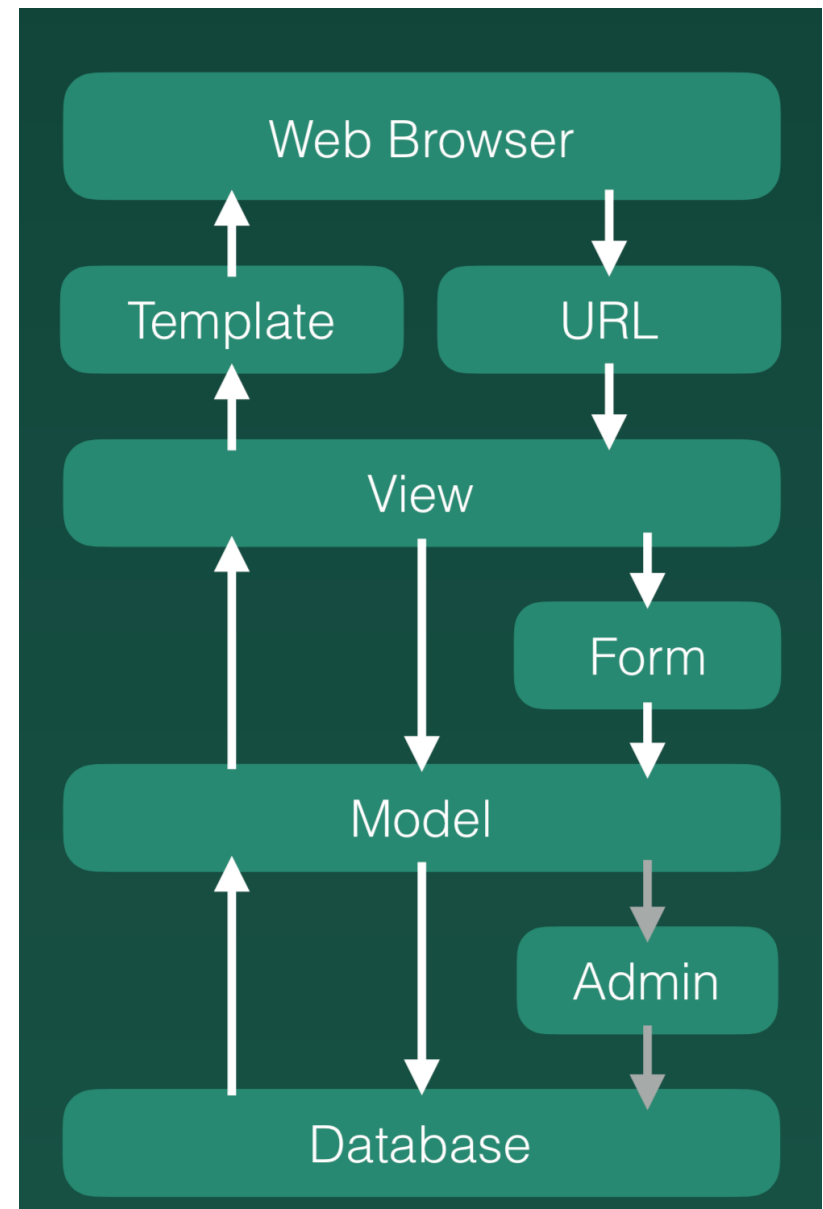
# The Model-View-Controller Architecture

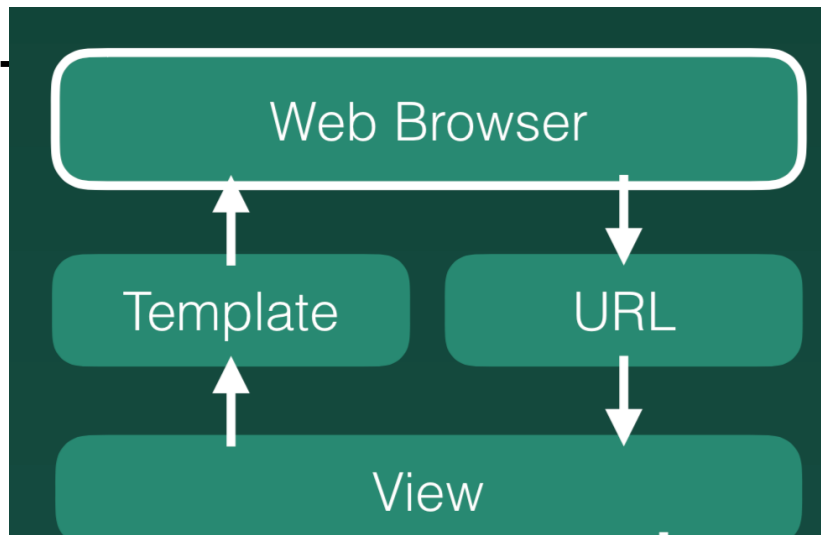
The **model-view-controller** architecture is a way to organize an application (not limited to the web!).

- The **model** is the data structure/repository.
- The **view** is the presentation/user interface.
- The **controller** is the logic that ties the model and view together and thus builds the application.

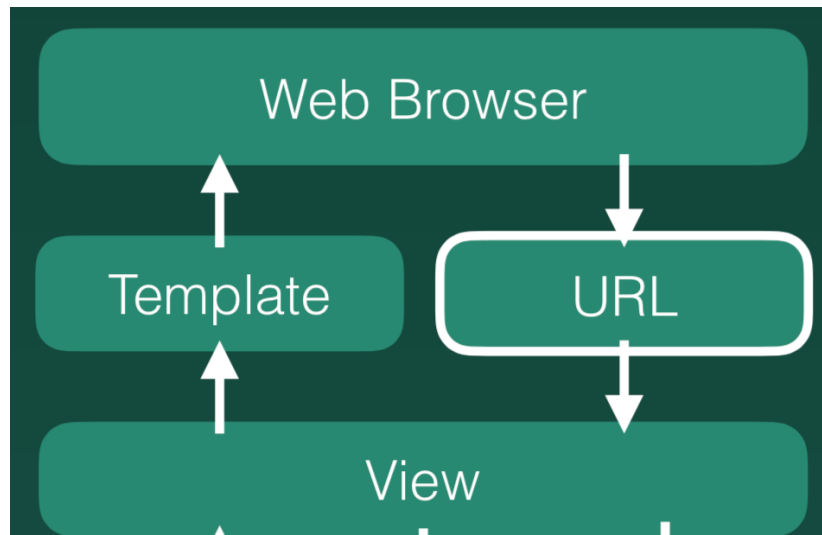
# Elements of a Django Web Application

- There are many components to a Django web application.
- The key is to learn which ones you can ignore!

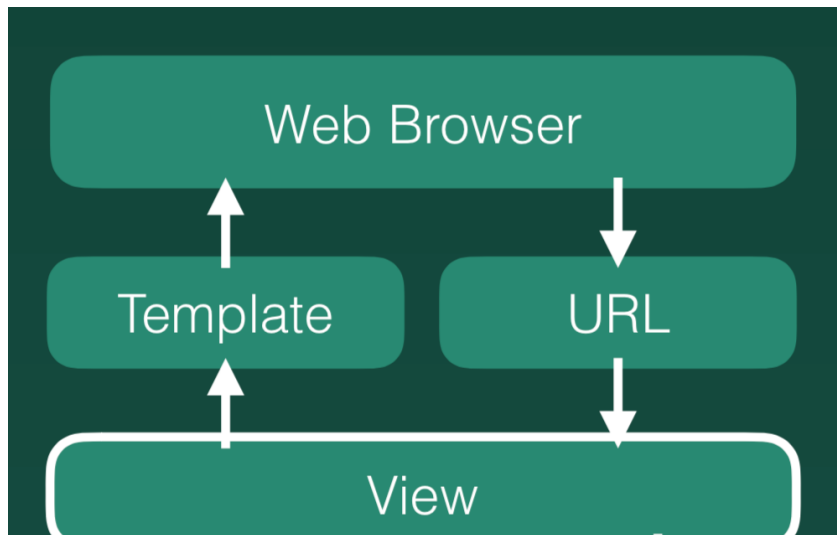




The **web browser** makes requests (by URL), and displays response as a graphical user interface.

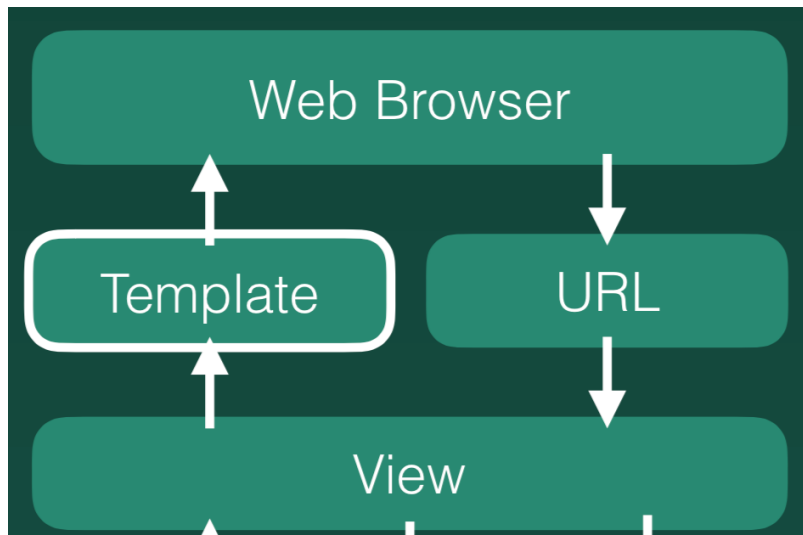


A **URL** is a way to locate a resource (e.g., a page or application).



A **View** is a program written in Python.

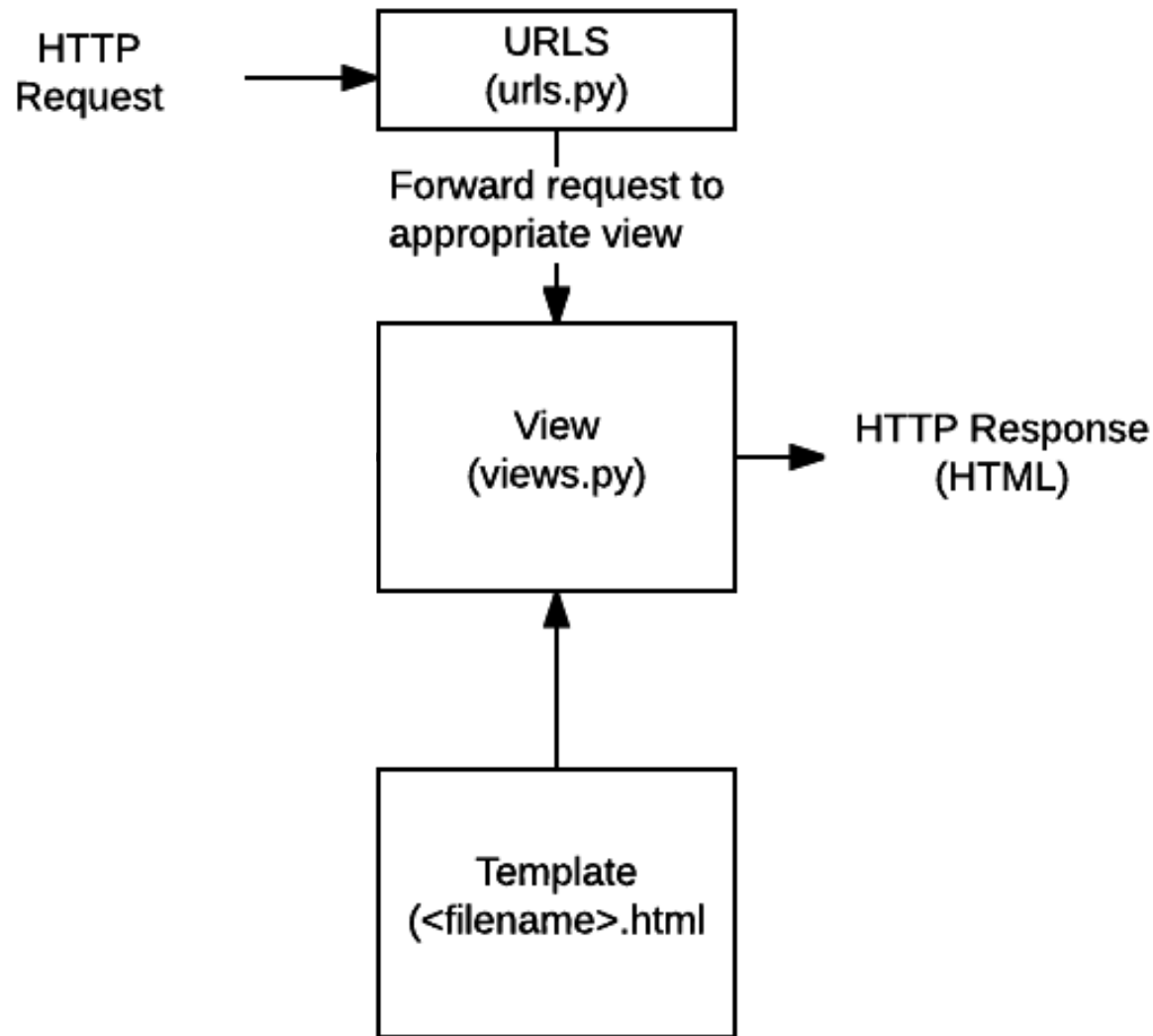
A view responds to a URL and creates an HTTP Response to send back to the client's browser.



A **Template** is an HTML page, with a twist.

The template mixes plain HTML with **scriptlets** – little bits of code that help display data that is generated by the View.

# The Workflow of a Django Web Application



# Reading Django Error Messages

## The Django Debug Page (in the browser)

- When `DEBUG = True`, Django shows a detailed error page instead of a crash
- Top of the page: the exception type (e.g., `TemplateDoesNotExist`) and message
- Below that: the full Python traceback — read it bottom-up to find your code
- Scroll down for request details: URL, GET/POST data, settings values

## The Terminal / Console Output

- Every request is logged: method, URL, and HTTP status code (200, 404, 500...)
- Python errors print a traceback here — check both the browser and terminal
- A `SyntaxError` in your code will prevent the server from starting at all
- Tip: keep the terminal visible while you work — it is your first source of truth

# Summary

- Create a project with `django-admin startproject`; add apps with `manage.py startapp`
- Run a local dev server with `python manage.py runserver` (port 8000 by default)
- Use Git and GitHub for version control; push to Gradescope and csa2 for deployment
- Virtual environments isolate project dependencies — always activate before working
- Django follows the MVC pattern: Models (data), Views (logic), Templates (presentation)
- Key files: `settings.py` (config), `urls.py` (routing), `views.py` (logic), `templates/` (HTML)